LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

Disusun Oleh :

Nama	: Diki Hurrisyail
NIM	: (124140088)
Kelas Praktikum	: RA
Mata Kuliah	: Pemrograman Berorientasi Objek

Dosen Pengampu:

Arkham Zahri Rakhman, S.Kom., M.Eng.

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNOLOGI INDUSTRI

INSTITUT TEKNOLOGI SUMATERA

2026

I. Tujuan Praktikum

Setelah menyelesaikan praktikum pada bab ini, Mahasiswa diharapkan mampu:

1. Menjelaskan konsep polymorphism dalam OOP
2. Memahami perbedaan method overloading dan method overriding
3. Mengimplementasikan method overriding pada subclass
4. Menggunakan polymorphism dalam program Java
5. Memahami manfaat polymorphism dalam pengembangan program

II. Alat dan Bahan

Pada praktikum ini alat dan bahan yang digunakan terdiri dari:

1. Laptop Windows 11: Sebagai perangkat utama untuk menjalankan, menulis, dan menguji program selama praktikum berlangsung.
2. JDK (Java Development Kit): Sebagai tools utama untuk mengembangkan, meng-compile, dan menjalankan program berbasis bahasa Java.
3. Visual Studio Code: Sebagai text editor atau IDE untuk menulis, mengedit, dan mengelola kode program Java dengan lebih mudah.
4. Modul Praktikum PBO BAB 4: Sebagai panduan dalam memahami materi serta langkah-langkah yang harus dilakukan selama praktikum.

III. Langkah Percobaan

Pada praktikum ini percobaan yang dilakukan seperti yang tertera pada subbab **5.12 Tugas Percobaan** di dalam modul. Berikut langkah percobaan yang dilakukan:

Percobaan 1:

Pada percobaan 1, dibuat class `Hewan` dengan method `suara()`, lalu subclass `Kucing`, `Anjing`, dan `Burung` yang meng-*override* method tersebut dengan suara yang berbeda sesuai jenis hewannya.

Kode Program:

```
class Hewan {
    void suara() {
        System.out.println("Hewan bersuara");
    }
}

class Kucing extends Hewan {
    void suara() {
        System.out.println("Mengeong");
    }
}

class Anjing extends Hewan {
    void suara() {
        System.out.println("Menggonggong");
    }
}

class Burung extends Hewan {
    void suara() {
        System.out.println("Berkicau");
    }
}
```

```
public class Main1 {
    public static void main(String[] args) {

        Hewan h1 = new Kucing();
        h1.suara();

        Hewan h2 = new Anjing();
        h2.suara();

        Hewan h3 = new Burung();
        h3.suara();
    }
}
```

Percobaan 2:

Pada percobaan 2, dibuat array bertipe `Hewan` yang berisi beberapa objek `Kucing`, `Anjing`, dan `Burung`, lalu digunakan perulangan untuk memanggil method `suara()` sehingga semua hewan dapat menampilkan suaranya dengan konsep polymorphism.

Kode Program:

```
public class Main2 {
    public static void main(String[] args) {
        Hewan[] daftarHewan = {
            new Kucing(),
            new Kucing(),
            new Anjing(),
            new Anjing(),
            new Burung()
        };

        for (Hewan h : daftarHewan) {
            h.suara();
        }
    }
}
```

```
class Hewan {
    void suara() {
        System.out.println("Hewan bersuara");
    }
}

class Kucing extends Hewan {
    void suara() {
        System.out.println("Mengeong");
    }
}

class Anjing extends Hewan {
    void suara() {
        System.out.println("Menggonggong");
    }
}

class Burung extends Hewan {
    void suara() {
        System.out.println("Berkicau");
    }
}
```

Percobaan 3:

Pada percobaan 3, setiap subclass ditambahkan method khusus seperti `berburu()`, `bermain()`, dan `terbang()`, kemudian method `suara()` dipanggil secara polymorphism dan method khusus dipanggil sesuai jenis objeknya.

Kode Program:

```
class Hewan {
    void suara() {
        System.out.println("Hewan bersuara");
    }
}

class Kucing extends Hewan {
    void suara() {
        System.out.println("Mengeong");
    }

    void berburu() {
        System.out.println("Kucing berburu tikus");
    }
}

class Anjing extends Hewan {
    void suara() {
        System.out.println("Menggonggong");
    }

    void bermain() {
        System.out.println("Anjing bermain bola");
    }
}

class Burung extends Hewan {
    void suara() {
        System.out.println("Berkicau");
    }

    void terbang() {
        System.out.println("Burung terbang di langit");
    }
}
```

```
public class Main3 {
    public static void main(String[] args) {
        Hewan[] daftarHewan = {
            new Kucing(),
            new Anjing(),
            new Burung()
        };

        System.out.println(" Suara ");
        for (Hewan h : daftarHewan) {
            h.suara();
        }

        System.out.println("\n Aktivitas");
        Kucing k = new Kucing();
        Anjing a = new Anjing();
        Burung b = new Burung();

        k.berburu();
        a.bermain();
        b.terbang();
    }
}
```

Percobaan 4:

Pada percobaan 4, dibuat class `OperasiMatematika` dengan beberapa method `tambah()` yang memiliki parameter berbeda untuk menunjukkan konsep method overloading, lalu setiap method dipanggil untuk menampilkan hasil perhitungannya.

Kode Program:

```
class OperasiMatematika {

    int tambah(int a, int b) {
        return a + b;
    }

    int tambah(int a, int b, int c) {
        return a + b + c;
    }

    double tambah(double a, double b) {
        return a + b;
    }
}
```

```
public class Main4 {
    public static void main(String[] args) {
        OperasiMatematika op = new OperasiMatematika();

         System.out.println("Operasi Tambah (2, 3)= " +
op.tambah(2, 3));
        System.out.println("Operasi Tambah (1, 2, 3)= " +
op.tambah(1, 2, 3));
         System.out.println("Operasi Tambah (2.5, 3.5)= "
+ op.tambah(2.5, 3.5));
    }
}
```

Percobaan 5:

Pada percobaan 5, dibuat program sistem data hewan dengan class `Hewan` sebagai superclass dan `Kucing`, `Anjing`, serta `Burung` sebagai subclass yang menggunakan constructor dan overriding method `suara()`, kemudian objek disimpan dalam array dan ditampilkan menggunakan perulangan.

Kode Program:

```
class Hewan {
    String nama;
    String jenis;

    Hewan(String nama, String jenis) {
        this.nama = nama;
        this.jenis = jenis;
    }

    String suara() {
        return "Hewan bersuara";
    }

    void tampilkanData() {
        System.out.println("Nama: " + nama +
                           " | Jenis: " + jenis +
                           " | Suara: " + suara());
    }
}

class Kucing extends Hewan {
    Kucing(String nama) {
        super(nama, "Kucing");
    }

    String suara() {
        return "Mengeong";
    }
}
```

```
class Anjing extends Hewan {
    Anjing(String nama) {
        super(nama, "Anjing");
    }

    String suara() {
        return "Menggonggong";
    }
}

class Burung extends Hewan {
    Burung(String nama) {
        super(nama, "Burung");
    }

    String suara() {
        return "Berkicau";
    }
}
```

```
public class Main5 {
    public static void main(String[] args) {
        Hewan[] daftarHewan = {
            new Kucing("Milo"),
            new Anjing("Buddy"),
            new Burung("Rio"),
            new Kucing("Luna"),
            new Anjing("Rocky")
        };

        System.out.println("Data Hewan");
        for (Hewan h : daftarHewan) {
            h.tampilkanData();
        }
    }
}
```

IV. Hasil Output Program

Percobaan 1:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan5 (main)
$ java -cp bin Main1
Mengeong
Menggonggong
Berkicau
```

Percobaan 2:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan5 (main)
$ java -cp bin Main2
Mengeong
Mengeong
Menggonggong
Menggonggong
Berkicau
```

Percobaan 3:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan5 (main)
$ java -cp bin Main3
 Suara
Mengeong
Menggonggong
Berkicau

 Aktivitas
Kucing berburu tikus
Anjing bermain bola
Burung terbang di langit
```

Percobaan 4:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan5 (main)
$ java -cp bin Main4
Operasi Tambah (2, 3)= 5
Operasi Tambah (1, 2, 3)= 6
Operasi Tambah (2.5, 3.5)= 6.0
```

Percobaan 5:

```
Pongo@hurey MINGW64 /d/Kuliah/Tugas/PBO/Praktikum/Kode/Pertemuan5 (main)
$ java -cp bin Main5
Nama: Milo | Jenis: Kucing | Suara: Mengeong
Nama: Buddy | Jenis: Anjing | Suara: Menggonggong
Nama: Rio | Jenis: Burung | Suara: Berkicau
Nama: Luna | Jenis: Kucing | Suara: Mengeong
Nama: Rocky | Jenis: Anjing | Suara: Menggonggong
```

V. Analisis Program

Pada percobaan 1 dan 2, program berfokus pada penerapan konsep dasar *polymorphism* melalui *method overriding*. Class `Hewan` dijadikan sebagai superclass yang memiliki method `suara()`, kemudian subclass seperti `Kucing`, `Anjing`, dan `Burung` mengubah perilaku method tersebut sesuai karakteristiknya. Pada percobaan 2, konsep ini dikembangkan dengan menyimpan berbagai objek dalam array bertipe `Hewan`, sehingga pemanggilan method `suara()` dapat dilakukan secara seragam menggunakan perulangan meskipun objek memiliki tipe yang berbeda.

Pada percobaan 3 dan 4, program mulai mengombinasikan beberapa konsep OOP sekaligus. Selain tetap menggunakan overriding, setiap subclass ditambahkan method khusus seperti `berburu()`, `bermain()`, dan `terbang()` yang hanya dimiliki oleh masing-masing class, sehingga diperlukan pengecekan tipe objek untuk memanggilnya. Sementara itu, percobaan 4 memperkenalkan *method overloading* pada class OperasiMatematika, di mana method dengan nama yang sama dapat memiliki parameter berbeda untuk menangani variasi input.

Pada percobaan 5, seluruh konsep yang telah dipelajari digabungkan dalam studi kasus sistem data hewan. Program menggunakan constructor untuk inisialisasi data, overriding untuk menentukan suara tiap hewan, serta array dan perulangan untuk menyimpan dan menampilkan beberapa objek secara efisien. Hasilnya, program mampu menampilkan informasi seperti nama, jenis, dan suara hewan secara terstruktur, yang mencerminkan penerapan konsep OOP secara menyeluruh.

VI. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep *polymorphism* dalam pemrograman berorientasi objek memungkinkan satu method memiliki perilaku yang berbeda meskipun menggunakan nama yang sama. Hal ini terjadi karena method tersebut dipanggil oleh objek yang memiliki tipe berbeda.

Dalam praktikum ini, polymorphism diterapkan melalui *method overriding* pada subclass `Kucing`, `Anjing`, dan `Burung` yang mengubah implementasi method `suara()` dari superclass `Hewan`. Selain itu, konsep *method overloading* juga diterapkan pada class `OperasiMatematika` melalui beberapa method `tambah()` dengan parameter yang berbeda-beda.

Pemanfaatan *inheritance* dan penggunaan referensi superclass memungkinkan berbagai objek disimpan dalam satu tipe yang sama dan diproses menggunakan perulangan. Dengan demikian, program menjadi lebih fleksibel, mudah dikembangkan, serta mendukung penggunaan kembali kode secara lebih efisien.